

HW3-1

2026.9.14 邓若辰 PB25111617

3.6

试证明：若借助栈由输入序列 $1, 2, \dots, n$ 得到的输出序列为 $p_1, p_2, \dots, p_n$ 。（它是输入序列的一个排列），则在输出序列中不可能出现这样的情形：存在着 $i < j < k$ 使 $p_j < p_k < p_i$ 。

假设存在 $i < j < k$ ，使得 $p_j < p_k < p_i$ 。

因 p_i 最大，而输入序列 $1, 2, \dots, n$ 递增，所以 p_i 在输入序列中最后入栈。又因 $i < j < k$ ， p_i 在输出序列中最先出栈。所以 p_j 和 p_k 必须在 p_i 入栈之前就已经入栈。又 $p_j < p_k$ ，且入栈顺序递增，所以 p_j 比 p_k 先入栈。因此在栈中， p_j 位于 p_k 的下方。随后 p_i 入栈并最先出栈。 p_i 出栈后，栈顶元素就是 p_k ，所以 p_k 必然比 p_j 先出栈。但 $j < k$ 要求 p_j 比 p_k 先出栈，矛盾。

故输出序列中不可能出现 $i < j < k$ 且 $p_j < p_k < p_i$ 的情形。

3.21

假设表达式由单字母变量和双目四则运算符构成，试写一个算法，将一个通常书写形式且书写正确的表达式转换为逆波兰式。

```
typedef enum {
    ADD,
    SUB,
    MUL,
    DIV,
    LEFT,
    RIGHT,
    END,
} OpType;

OpType getOpType(char c) {
    switch (c) {
        case '+': return ADD;
        case '-': return SUB;
        case '*': return MUL;
        case '/': return DIV;
        case '(': return LEFT;
        case ')': return RIGHT;
        case '#': return END;
    }
}

char PrecedeTable[7][7] = {
    //          02: '+'  '-'  '*'  '/'  '('  ')'  '#'
    /* 01: '+' */ { '>', '>', '<', '<', '<', '>', '>' },
    /* 01: '-' */ { '>', '>', '<', '<', '<', '>', '>' },
    /* 01: '*' */ { '>', '>', '>', '>', '<', '>', '>' },
    /* 01: '/' */ { '>', '>', '>', '>', '<', '>', '>' },
    /* 01: '(' */ { '<', '<', '<', '<', '<', '=', '>' },
```

```

/* 01: ')' */ { '>', '>', '>', '>', '>', '>', '>' },
/* 01: '#' */ { '<', '<', '<', '<', '<', '<', '=' }
};

char Precede(char c1, char c2) {
    OpType idx1 = getOpType(c1);
    OpType idx2 = getOpType(c2);
    return PrecedeTable[idx1][idx2];
}

void transformer(char* expression, char* output){
    int i=0;
    int j=0;
    char e;
    SqStack op;
    InitStack(&op);
    Push(&op, '#');
    while(*(expression+i)!='\0'){
        if(isalpha(*(expression+i))){
            *(output+j)=*(expression+i);
            j++;
            i++;
        }else{
            char cmp;
            GetTop(&op, &e);
            char c1=e;
            char c2=*(expression+i);
            cmp=Precede(c1, c2);
            if(cmp=='<'){
                Push(&op, c2);
                i++;
            }else if(cmp=='>'){
                Pop(&op, &e);
                *(output+j)=e;
                j++;
            }else if(cmp=='='){
                Pop(&op, &e);
                i++;
            }
        }
    }
    *(output+j)='\0';
}

```

3.22

如题3.21的假设条件，试写一个算法，对以逆波兰式表示的表达式求值

```

#假设单字母变量的计算范围均在int能表示的范围之内，并用数组存储连续的26个整型数值
#define ElemType double

```

```
int values[26] = {0};
double calculation(char* expression){
    SqStack num;
    InitStack(&num);
    int i=0;
    double e;
    while(*(expression+i)!='\0'){
        char ch=*(expression+i);
        if(isalpha(*(expression+i))){
            Push(&num,values[ch-'a']);
            i++;
        }else{
            Pop(&num,&e);
            double c2=e;
            Pop(&num,&e);
            double c1=e;
            double result=0;
            if(*(expression+i)=='+'){
                result=c1+c2;
            }
            if(*(expression+i)=='-'){
                result=c1-c2;
            }
            if(*(expression+i)=='*'){
                result=c1*c2;
            }
            if(*(expression+i)=='/'){
                if(c2==0){
                    exit(1);
                }
                result=c1/c2;
            }
            Push(&num,result);
            i++;
        }
    }
    double final;
    Pop(&num,&final);
    return final;
}
```